

Optimize Open-Headunit for Low RAM

<https://gemini.google.com/app/8bc5165ccdc71176>

User prompt: Take a look of the code of this repo <https://github.com/andreknieriem/open-headunit> is there possibility to enhance this app to lower gb devices 1 GB?

Response: Yes, **open-headunit** can absolutely be optimized to run reliably on devices with 1 GB of RAM (or lower).

Because the app operates primarily as an **H.264 video decoder** and byte stream proxy for the Android Auto protocol, its inherent architecture is quite lean. Memory bottlenecks on 1 GB devices usually stem from **buffer bloat**, **OpenGL surface rendering overhead**, **high resolution framebuffers**, and **Java GC spikes**.

Key Areas to Enhance for 1 GB RAM Support

1. Zero-Copy Video Decoding Pipeline

- **Use Direct SurfaceView Canvas:** Default to direct hardware surface rendering via SurfaceView instead of TextureView or standard OpenGL views. TextureView creates additional off-screen GPU framebuffers that consume valuable RAM and GPU memory.
- **Direct ByteBuffer Recycling:** Allocate video stream decoding buffers using `ByteBuffer.allocateDirect()` outside the JVM heap and reuse fixed-size ring buffers to minimize garbage collection (GC) pressure.

2. Protocol Stream & Audio Buffer Tweak

- **Reduce Socket Buffer Window:** Lower network/USB socket buffer sizes in the Android Auto protocol reader. Overly generous TCP buffers cause hidden memory bloat during high-bitrate video bursts.
- **Streamlined Audio Track:** Configure AudioTrack buffers to the absolute minimum required for continuous playback to prevent PCM queue accumulation.

3. Low Memory System Profiling

- **Enable Low-RAM Flags in AndroidManifest.xml:**

```
<application
    android:largeHeap="false"
    android:hardwareAccelerated="true"
    android:hasFragileUserData="false">
```
- **Aggressive Background Cleanup:** Release decoder resources (`MediaCodec.release()`) and clear protocol state immediately on `onStop()` or when the video stream loses focus.

Recommended Settings for 1 GB Devices

You can achieve smooth performance immediately by tweaking the app's settings:

- **Resolution:** Set to **800x480** or **480p** (avoid 720p/1080p, as higher resolutions require significantly larger framebuffers).
- **View Mode:** Set to **SurfaceView** (or GLES20 on Android 5.0–8.1 legacy hardware) instead of TextureView.
- **FPS:** Limit output to **30 FPS** to reduce memory bandwidth requirements.
- **Connection:** Prefer **Wired USB connection** over Wireless Helper/Wi-Fi Direct to eliminate background network stack memory overhead.

Sources

1. [Settings · andreknieriem/open-headunit Wiki - GitHub](#)

User prompt: Show code examples for optimizing MediaCodec direct byte buffer allocations and SurfaceView rendering in open-headunit.

Response: To optimize MediaCodec buffer handling and SurfaceView rendering for low-RAM devices in **open-headunit**, focus on

two primary objectives: eliminating Java GC allocations in the hot video stream loop and routing decoded video frames directly to hardware surfaces.

1. Zero-Copy SurfaceView Hardware Output

Instead of retrieving raw YUV byte arrays or rendering through a TextureView (which requires extra GPU off-screen framebuffers), pass the native Surface from SurfaceView directly into MediaCodec.

```
public class VideoDecoderEngine implements SurfaceHolder.Callback {
    private MediaCodec mediaCodec;
    private Surface surface;

    @Override
    public void surfaceCreated(SurfaceHolder holder) {
        this.surface = holder.getSurface();
        // Force 16-bit or direct RGB surface format to reduce hardware RAM footprint
        holder.setFormat(PixelFormat.RGB_565);
        initCodec();
    }

    private void initCodec() {
        try {
            MediaFormat format = MediaFormat.createVideoFormat(MediaFormat.MIMETYPE_VIDEO_AVC, 800, 480);

            // Low-latency and low-buffer flags for weak SoCs
            format.setInteger(MediaFormat.KEY_LOW_LATENCY, 1);
            format.setInteger(MediaFormat.KEY_MAX_INPUT_SIZE, 512 * 1024); // Cap max buffer to 512KB

            mediaCodec = MediaCodec.createDecoderByType(MediaFormat.MIMETYPE_VIDEO_AVC);

            // Passing surface directly enables zero-copy hardware decoding directly to screen
            mediaCodec.configure(format, surface, null, 0);
            mediaCodec.start();
        } catch (IOException e) {
            Log.e("Decoder", "Failed to initialize codec", e);
        }
    }

    @Override
    public void surfaceChanged(SurfaceHolder holder, int format, int w, int h) {}

    @Override
    public void surfaceDestroyed(SurfaceHolder holder) {
        if (mediaCodec != null) {
            mediaCodec.stop();
            mediaCodec.release();
            mediaCodec = null;
        }
    }
}
```

2. Direct ByteBuffer Pool for Incoming NAL Units

To prevent memory churn from continuous byte[] allocations when demuxing the raw Android Auto video stream, allocate off-heap direct byte buffers once and reuse them.

```
public class StreamBufferPool {
    private final ByteBuffer inputBuffer;
    private final int MAX_FRAME_SIZE = 512 * 1024; // 512 KB pre-allocated slot

    public StreamBufferPool() {
        // Allocated natively outside JVM heap to eliminate garbage collection (GC) pauses
        inputBuffer = ByteBuffer.allocateDirect(MAX_FRAME_SIZE);
    }

    public void feedVideoChunk(byte[] chunkData, int length, MediaCodec codec) {
        int inputIndex = codec.dequeueInputBuffer(10000 /* 10ms timeout */);
        if (inputIndex >= 0) {
            // Get the codec's direct input buffer
            ByteBuffer codecBuffer = codec.getInputBuffer(inputIndex);
```

```

        if (codecBuffer != null) {
            codecBuffer.clear();

            // Copy stream data directly without object creation
            codecBuffer.put(chunkData, 0, length);

            codec.queueInputBuffer(
                inputIndex,
                0,
                length,
                System.nanoTime() / 1000,
                0
            );
        }
    }
}

```

3. Direct Frame Release Loop

Ensure output buffers are freed instantly back to the Android display subsystem (Surface) as soon as they are decoded:

```

public void renderDecodedFrames(MediaCodec codec) {
    MediaCodec.BufferInfo bufferInfo = new MediaCodec.BufferInfo();

    // Dequeue without blocking the UI thread
    int outputIndex = codec.dequeueOutputBuffer(bufferInfo, 0);

    while (outputIndex >= 0) {
        // Passing 'true' renders the buffer directly to the SurfaceView and frees memory instantly
        codec.releaseOutputBuffer(outputIndex, true);

        outputIndex = codec.dequeueOutputBuffer(bufferInfo, 0);
    }
}

```

User prompt: Show how to optimize audio PCM buffer allocation and AudioTrack queuing for 1 GB Android Auto devices.

Response: Optimizing PCM audio playback for Android Auto on 1 GB devices requires configuring AudioTrack for low-latency fast-path routing, using a fixed circular buffer, and avoiding byte array allocations in the hot audio stream loop.

1. Low-Latency AudioTrack Initialization

Set native 48 kHz output with hardware flags to bypass the system mixer's resampler and prevent internal memory accumulation.

```

public class LowMemoryAudioPlayer {
    private AudioTrack audioTrack;
    private int minBufferSize;

    public void initAudioTrack() {
        int sampleRate = 48000; // Android Auto standard PCM rate
        int channelConfig = AudioFormat.CHANNEL_OUT_STEREO;
        int audioFormat = AudioFormat.ENCODING_PCM_16BIT;

        // Calculate minimum buffer size needed for hardware DMA
        minBufferSize = AudioTrack.getMinBufferSize(sampleRate, channelConfig, audioFormat);

        AudioAttributes attributes = new AudioAttributes.Builder()
            .setUsage(AudioAttributes.USAGE_MEDIA)
            .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
            .setFlags(AudioAttributes.FLAG_LOW_LATENCY) // Bypasses deep-buffer audio routes
            .build();

        AudioFormat format = new AudioFormat.Builder()
            .setSampleRate(sampleRate)
            .setChannelMask(channelConfig)
            .setEncoding(audioFormat)
            .build();
    }
}

```

```

        // 2x minBufferSize enables double-buffering with minimal RAM usage
        audioTrack = new AudioTrack.Builder()
            .setAudioAttributes(attributes)
            .setAudioFormat(format)
            .setBufferSizeInBytes(minBufferSize * 2)
            .setTransferMode(AudioTrack.MODE_STREAM)
            .setPerformanceMode(AudioTrack.PERFORMANCE_MODE_LOW_LATENCY)
            .build();

        audioTrack.play();
    }
}

```

2. Zero-Allocation Circular PCM Buffer

Instead of instantiating new `byte[]` arrays every time a USB/network frame arrives, write audio packets into a single pre-allocated ring buffer.

```

public class PcmRingBuffer {
    private final byte[] buffer;
    private int head = 0;
    private int tail = 0;
    private int available = 0;

    public PcmRingBuffer(int capacityBytes) {
        // Pre-allocate once on startup
        this.buffer = new byte[capacityBytes];
    }

    public synchronized boolean write(byte[] data, int length) {
        if (buffer.length - available < length) {
            // Drop oldest data on overflow to prevent latency drift and memory spikes
            head = (head + length) % buffer.length;
            available -= length;
        }

        for (int i = 0; i < length; i++) {
            buffer[tail] = data[i];
            tail = (tail + 1) % buffer.length;
        }
        available += length;
        notifyAll();
        return true;
    }

    public synchronized int read(byte[] outBuffer, int length) {
        while (available == 0) {
            try {
                wait(50);
                if (available == 0) return 0;
            } catch (InterruptedException e) {
                return 0;
            }
        }

        int bytesToRead = Math.min(length, available);
        for (int i = 0; i < bytesToRead; i++) {
            outBuffer[i] = buffer[head];
            head = (head + 1) % buffer.length;
        }
        available -= bytesToRead;
        return bytesToRead;
    }
}

```

3. Non-Blocking Audio Queue Loop

Stream audio continuously in non-blocking mode to keep the thread context switching light and eliminate GC pauses.

```
public class AudioPlaybackWorker implements Runnable {
    private final PcmRingBuffer ringBuffer = new PcmRingBuffer(64 * 1024); // Fixed 64KB pool
    private final byte[] readChunk = new byte[2048]; // Single reused chunk array
    private boolean isRunning = true;

    @Override
    public void run() {
        android.os.Process.setThreadPriority(android.os.Process.THREAD_PRIORITY_URGENT_AUDIO);

        while (isRunning) {
            int readBytes = ringBuffer.read(readChunk, readChunk.length);
            if (readBytes > 0) {
                // Non-blocking write prevents the playback thread from hanging on slow audio HALs
                audioTrack.write(readChunk, 0, readBytes, AudioTrack.WRITE_NON_BLOCKING);
            }
        }
    }
}
```

User prompt: Do that optimization for low spec units will help other units?

Response: **Yes, absolutely.** Optimizing the video and audio pipeline for low-spec hardware benefits higher-spec devices just as much. While high-RAM devices can easily mask inefficient code with raw compute power, applying zero-copy rendering and allocation-free streaming yields noticeable improvements on any hardware:

- **Lower Touch & Display Latency:** Eliminating intermediate byte array copies and off-screen framebuffers shortens the pipeline from USB input to screen display, making touch interactions feel significantly snappier.
- **Zero Micro-Stuttering:** Removing object allocations from the hot video/audio loop prevents the JVM Garbage Collector (GC) from running. This eliminates random frame drops and audio pops during high-bitrate bursts.
- **Reduced Thermal Throttling:** Less CPU/GPU overhead means less heat generation. Automotive head units operate in hot dashboard environments where high-spec SoCs often aggressively throttle performance during long drives.
- **Background Service Stability:** Keeping memory consumption under 100 MB prevents Android's Low Memory Killer (LMK) from terminating background processes like Bluetooth listeners, reverse camera triggers, or steering wheel control services.

Architectural Optimizations vs. Hard Constraints

To get the best of both worlds, distinguish between **code architecture** and **stream configurations**:

Optimization Type	Action for High-Spec Units
Architectural (<i>Direct ByteBuffer, SurfaceView, AudioTrack low-latency mode</i>)	Apply globally. Every device benefits from zero-copy memory handling and GC elimination.
Stream Constraints (<i>Resolutions like 480p, 30 FPS limits, low audio buffer sizes</i>)	Make configurable. Allow high-spec units to request 720p/1080p at 60 FPS while keeping the lean zero-copy underlying pipeline.

By keeping the core architecture zero-copy and exposing stream resolution/bitrate as settings, lower-end devices get smooth 480p/30 FPS playback while higher-end units get crisp 1080p/60 FPS performance without any micro-stuttering.

User prompt: How does <https://github.com/moonlight-stream/moonlight-android/> handles video decoding, do it resembles to open headunit? Open headunit is experiencing decoding artifacts an I'm getting out of ideas on how to solve it

Response: Yes, **Moonlight-Android and Open-Headunit fundamentally share the same end goal:** real-time, low-latency H.264/HEVC stream decoding via Android's MediaCodec framework. However, **how they handle the stream architecture is drastically different**, which explains why Moonlight is exceptionally stable while Open-Headunit often suffers from visual artifacts (green/corrupted frames, tearing, macroblocking, or grey smears).

How Moonlight Handles Decoding vs. Open-Headunit

Architectural Aspect	Open-Headunit	Moonlight Android
Layering	Standard Java API (MediaCodec Java wrapper).	NDK / Native C Code (AMediaCodec or direct NDK bindings via moonlight-common-c).
NAL Unit Parsing	Relies on simple byte searching/splitting in Java before queuing to codec.	Strict Native Parser (Libavcodec / ffmpeg integration) to ensure complete NAL boundaries and NAL prefix formatting (0x00000001).
SPS / PPS Handling	Passes SPS/PPS once or inline without re-configuring when corrupt frames hit.	Explicit Codec Invalidation & IDR Force: Intercepts frame loss signals and re-injects SPS/PPS in Annex-B format whenever a drop occurs.
Buffer Allocation	Managed by JVM Garbage Collector and Standard Java ByteBuffer.	Native direct memory mapping (ANativeWindow + C-level ring buffers) avoiding GC stalls completely.

Why Open-Headunit Experiences Decoding Artifacts

Video decoding artifacts (macroblocking, missing I-frames, split color blocks, or green flickering) on Android Auto head units usually stem from **four specific root causes**:

1. Corrupted / Split NAL Units over USB/Wi-Fi

Android Auto sends video split across RTP/accessory protocol packets. If Open-Headunit feeds incomplete NAL units or missing H.264 start codes (0x00 0x00 0x00 0x01) into MediaCodec.queueInputBuffer(), the hardware decoder tries to process partial P-frames, causing severe macroblocking until the next Keyframe (I-frame).

2. SPS / PPS Parameter Sets Missing Annex-B Headers

Low-cost Android head unit chipsets (Allwinner, Rockchip, MediaTek) are notoriously pickier than Qualcomm Snapdragon SoCs. They fail silently if SPS (Sequence Parameter Set) and PPS (Picture Parameter Set) lack explicit Annex-B start codes or if BUFFER_FLAG_CODEC_CONFIG is improperly set on initialization.

3. Buffer Overruns and Dropped TCP/USB Packets

If Java GC pauses the thread for even 20 ms, incoming USB packets overflow the socket buffer. The driver drops video chunks, corrupting the slice structure of the current frame.

Concrete Fixes for Open-Headunit Artifacts

Fix 1: Ensure Strict Annex-B Formatting and Codec Config Buffers

Before queuing the first video frame, explicitly pass SPS and PPS inside csd-0 and csd-1 format keys, or prepend them directly to keyframes with the 0x00000001 prefix:

```
// Pass SPS and PPS explicitly via MediaFormat during setup
MediaFormat format = MediaFormat.createVideoFormat(MediaFormat.MIMETYPE_VIDEO_AVC, width, height);

byte[] sps = { 0x00, 0x00, 0x00, 0x01, 0x67, ... }; // Extract from Android Auto handshake
byte[] pps = { 0x00, 0x00, 0x00, 0x01, 0x68, ... };

format.setByteBuffer("csd-0", ByteBuffer.wrap(sps));
format.setByteBuffer("csd-1", ByteBuffer.wrap(pps));

// Set low-latency flag for Vendor Decoders
format.setInteger(MediaFormat.KEY_LOW_LATENCY, 1);
format.setInteger("vendor.low-latency.enable", 1); // Specific to Rockchip / Allwinner
```

Fix 2: Flush Codec Buffer on Frame Drop / Keyframe Re-Sync

When packet loss or a corrupt frame boundary is detected, flush MediaCodec immediately to prevent reference frame corruption:

```
public void handleFrameLoss() {
    if (mediaCodec != null) {
        // Clear internal decoder queues so old corrupt P-frames don't distort upcoming frames
        mediaCodec.flush();
    }
}
```

Fix 3: Force Hardware Vendor Decoder Selection

By default, `MediaCodec.createDecoderByType("video/avc")` might fall back to Google's software decoder (`OMX.google.h264.decoder`), which struggles at high bitrates, or choose an unstable vendor wrapper. Inspect available decoders and explicitly prioritize non-Google hardware decoders:

```
public static String getBestDecoderName(String mimeType) {
    MediaCodecList list = new MediaCodecList(MediaCodecList.ALL_CODECS);
    for (MediaCodecInfo info : list.getCodecInfos()) {
        if (info.isEncoder()) continue;
        for (String type : info.getSupportedTypes()) {
            if (type.equalsIgnoreCase(mimeType)) {
                String name = info.getName();
                // Avoid Google SW decoder for real-time video streaming
                if (!name.startsWith("OMX.google.") && !name.startsWith("c2.android.")) {
                    return name; // e.g., OMX.rk.video_decoder.avc or OMX.MTK.VIDEO.DECODER.AVC
                }
            }
        }
    }
    return null; // Fallback to default
}
```

Fix 4: Set Adaptive Playback and Disable Surface Smoothing

Disable surface smoothing in `MediaFormat` so the hardware decoder outputs frames instantly without waiting for a re-order queue:

```
// Disables internal B-frame buffer delay
format.setInteger(MediaFormat.KEY_MAX_INPUT_SIZE, 1024 * 512);
format.setInteger("priority", 0); // Real-time priority
```

User prompt: As open headunit can't request IDR, do those proposed fixes would work? Do h264 will benefit from them? Can It be harmful for KitKat units?

Response: 1. Will the proposed fixes work without IDR requests?

`mediaCodec.flush()` is **HARMFUL without IDR requests**, but the other fixes will significantly benefit H.264 decoding.

When Open-Headunit cannot request a new IDR frame (keyframe) from the phone:

- **`mediaCodec.flush()` will make corruption worse:** Calling `flush()` wipes the decoder's internal Decoded Picture Buffer (DPB). If you flush the decoder mid-stream without receiving an immediate IDR frame, the decoder receives subsequent P-frames without their reference images. This turns temporary pixelation into a **permanent green/gray screen freeze** until the phone happens to send its next periodic keyframe.
- **Annex-B NAL assembly & SPS/PPS csd-0 initialization WORK:** Ensuring every packet starts with `0x00000001` and passing explicit sequence parameter sets (SPS/PPS) during `MediaFormat` setup prevents corrupted frames from entering the decoder in the first place. Preventing initial parsing errors reduces the need for keyframe recovery.

2. Can these fixes be harmful to Android 4.4 KitKat units?

Yes. Applying Lollipop+ (API 21) `MediaCodec` methods blindly will **crash or break** Android 4.4 KitKat (API 19) head units.

Critical KitKat Pitfalls:

1. **`getInputBuffer(index)` Crash (`NoSuchMethodError`):** Direct buffer methods (`codec.getInputBuffer(index)`) were

introduced in API 21. Calling them on KitKat causes an immediate app crash. KitKat requires legacy buffer array fetching:

```
ByteBuffer inputBuffer;
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
    inputBuffer = codec.getInputBuffer(index); // API 21+
} else {
    inputBuffer = codec.getInputBuffers()[index]; // KitKat (API 19) legacy array
}
```

2. **MediaCodecList Constructor Crash:** new MediaCodecList(MediaCodecList.ALL_CODECS) does not exist on API 19. You must use static iterations for KitKat:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
    MediaCodecList list = new MediaCodecList(MediaCodecList.ALL_CODECS);
    // Iterate list.getCodecInfos()
} else {
    int count = MediaCodecList.getCodecCount(); // KitKat static API
    for (int i = 0; i < count; i++) {
        MediaCodecInfo info = MediaCodecList.getCodecInfoAt(i);
    }
}
```

3. **Vendor HAL Rejection of Low-Latency Keys:** KEY_LOW_LATENCY ("low-latency") was formally introduced in Android 11. Passing unknown vendor keys on legacy Rockchip (RK3066/RK3188) or Allwinner KitKat ROMs can cause mediaCodec.configure() to fail with an IllegalArgumentException. Always wrap format flags in conditional checks (Build.VERSION.SDK_INT >= 30).

Sources

1. [MediaCodec - Android SDK - iut-fbleau.fr](#)

User prompt: Explore the android auto protocol and see if we can request IDR frames

Response: The official Android Auto Protocol (AAP) specification **does not contain a dedicated "Request IDR Frame" or "Request Keyframe" message.**

However, **you can force the phone's encoder to transmit an immediate IDR frame by cycling the AAP Video Focus state.**

How the AAP Video Pipeline Handles Focus

In the Android Auto Protocol (AASDK / AAP), video streaming is managed by the VideoChannel through **Video Focus** messages (VideoFocusIndication and VideoFocusRequest):

- **VIDEO_FOCUS_PROJECTED (Active):** The head unit is displaying the phone's UI. The phone continuously encodes and streams H.264 P-frames.
- **VIDEO_FOCUS_UNFOCUSED / VIDEO_FOCUS_NATIVE (Inactive):** Sent when switching to a native car menu or backup camera. The phone pauses or drops its video encoder stream.

When Video Focus transitions back to VIDEO_FOCUS_PROJECTED, the Android Auto app on the phone is required to re-initialize or resume its video encoder session. Because a newly resumed stream cannot be decoded without reference data, **the phone's MediaCodec encoder automatically generates a fresh SPS/PPS header and an IDR keyframe as the very first packet.**

Protocol Sequence to Force an IDR Frame

To force an IDR frame programmatically from the head unit without dropping the USB/Wi-Fi connection:

1. **Send Unfocused State:** Transmit a VideoFocusIndication message with mode = VIDEO_FOCUS_UNFOCUSED (or request a temporary focus release).
2. **Short Delay:** Wait ~50–100 ms for the phone's MediaService to register the state transition.
3. **Send Projected State:** Transmit a VideoFocusIndication message with mode = VIDEO_FOCUS_PROJECTED (and reason = 1 / User action).
4. **Receive Keyframe:** The next incoming video packet from the phone will contain the H.264 NAL start codes for SPS (0x67), PPS

(0x68), and an IDR frame (0x65).

Implementation Code Pattern (Java / AAP Channel Service)

```
public class VideoChannelHandler {
    private long lastKeyframeRequestTime = 0;
    private static final long KEYFRAME_LOCKOUT_MS = 1000; // Debounce threshold

    public void triggerHardwareIDR() {
        long now = System.currentTimeMillis();
        // Prevent spamming focus changes which could crash legacy HALs
        if (now - lastKeyframeRequestTime < KEYFRAME_LOCKOUT_MS) {
            return;
        }
        lastKeyframeRequestTime = now;

        // Step 1: Temporarily signal loss of video focus
        sendVideoFocusIndication(VideoFocusMode.VIDEO_FOCUS_UNFOCUSED);

        // Step 2: Re-request active video focus after a short delay
        handler.postDelayed(() -> {
            sendVideoFocusIndication(VideoFocusMode.VIDEO_FOCUS_PROJECTED);
        }, 50);
    }

    private void sendVideoFocusIndication(VideoFocusMode mode) {
        VideoFocusIndicationData.Builder builder = VideoFocusIndicationData.newBuilder();
        builder.setFocusMode(mode);
        builder.setUnfocusedReason(0);

        byte[] payload = builder.build().toByteArray();
        sendProtocolFrame(CHANNEL_VIDEO, MESSAGE_VIDEO_FOCUS_INDICATION, payload);
    }
}
```

Important Considerations for Head Units

- **Debouncing & Lockout:** Never trigger focus cycling on every dropped P-frame. Implement a **Keyframe Lockout Window** (500 ms – 1000 ms) so that network jitter does not cause rapid back-to-back focus toggles, which can crash the head unit's video decoder.
 - **Decoder Flush Handling:** When triggering a focus-driven IDR, flush the local hardware decoder (`mediaCodec.flush()`) **only after** receiving the new SPS/PPS header from the phone to avoid blank screens.
-